COMP 018: Database Administration - Activity 4 Progress Report
Access Control, Database Users, Roles, Privileges, and Security Governance

Project Identification & Metadata

Course	: COMP 018 - Database Administration
Evaluating Group	: Group 4: Olivarez, Abangan, Pocsidio, Infantado, and Bartolata
Evaluating Group Repository	: https://github.com/Nycri/brewpos
Target Project Monitored	: Park-It: Parking Management System (Park-It Database Management System)
Target Project Group	: Group 2: Miller Devera, Jhon Francis, et al.
Target Project Repository	: https://github.com/millerdevera6-glitch/park-it-database
Primary Database	: grp_parkitdb (PostgreSQL 18.6 on x86_64 Windows, compiled by MSVC)
Restored Test Database	: grp_parkitdb_restore_test

ACTIVITY 4: Database Users, Roles, Privileges, and Access Control

1.1 Access-Control Plan & Architecture

In default database setups, web backends frequently connect using the PostgreSQL superuser account. Running an interactive web application under an administrative superuser account severely violates the Principle of Least Privilege. If an unauthenticated user exploits an application vulnerability or API endpoint, they possess unrestricted data definition and manipulation capabilities across the database cluster, enabling them to drop tables, modify payment records, or alter server settings.

To secure Group 2's Park-It system, a role-based access control architecture was implemented to decouple application runtime operations, compliance reporting, and administrative database maintenance. The architecture defines non-login group roles to manage privilege templates alongside dedicated login accounts assigned to their corresponding group roles.

Under this plan, the application runtime account holds read, insert, and update capabilities across active tables, but its row deletion rights are strictly confined to parking zones to support lot decommissioning API calls. Deletion privileges are strictly denied on transaction logs, payment records, vehicle histories, and staff user profiles. In addition, the read-only auditing account receives query access for metric inspection but is denied access to staff user profile tables, safeguarding employee password hashes from exposure.

1.2 Access-Control Role Definitions and Privilege Structure

The application runtime user operates under a dedicated group role assigned to handle parking zone operations, vehicle check-ins, occupancy counters, and checkout billing. Its permissions maintain full selection, insertion, and update capabilities on operational tables while confining deletion strictly to parking zone records, protecting transactional logs and billing histories from

deletion.

The read-only auditor account operates through a compliance group role to review parking occupancy, vehicle transaction logs, and daily payment receipts. It holds read access across operational tables while omitting read permissions on staff credentials. The backup operator account is assigned dedicated read permissions across tables and sequences to support logical dump generation without write or schema modification rights. Database administration tasks remain reserved for the database administrator superuser account.

1.3 Access-Control Configuration Summary
The setup process established non-login group roles representing runtime, read-only audit, and backup profiles, followed by the creation of matching login accounts with dedicated authentication credentials. Access privileges were revoked from public roles across the database and public schema to enforce explicit role grants. Connection privileges and schema usage rights were assigned to the designated group roles, explicit table grants were configured to enforce least-privilege policies, and default schema privileges were established to preserve access rules across future database objects.

1.4 Verification Results: Runtime Account Testing
Verification testing demonstrated that the runtime account successfully queries active parking zones, inserts new zone records while advancing sequence counters, updates occupancy metrics, and deletes decommissioned parking zones. When row deletion operations were attempted on transaction records or staff user identities, the database engine denied execution under the narrow deletion policy. Data definition commands, such as dropping tables, were rejected due to missing object ownership permissions.

1.5 Verification Results: Read-Only Auditor Account Testing
Inspection testing confirmed that the read-only auditing account successfully executes multi-table queries joining parking zones, vehicles, and transaction logs. When query attempts were made on staff credential tables containing authentication hashes, access was denied by the database security layer. Attempted modification operations, including inserting new zones, updating capacity limits, or deleting records, were uniformly blocked by the engine.

1.6 Verification Results: Cross-Database & Administrative Boundary Testing
Administrative isolation testing showed that database creation commands attempted under application accounts were denied due to missing administrative creation privileges. Connection attempts to unrelated databases on the server instance failed because public connection permissions were revoked, confining access strictly to authorized targets.

1.7 Issues, Fixes, and Feedback to Owners
During deployment, default superuser connection strings in configuration files were replaced with least-privilege role credentials. Sample seed data in the local environment was expanded to allow testing across multi-table joins and billing transactions. Environment variable keys were

standardized to ensure consistent initialization.

Feedback provided to Group 2 commended the well-designed relational schema and strict constraint checks. Recommendations advised updating default configuration examples to reference least-privilege application accounts instead of superuser credentials, as well as expanding database seed scripts to include sample vehicles, transactions, and staff accounts out-of-the-box.

CLASS GOOGLE SHEETS TRACKER ENTRY (Activity 4)

Column Header	Value
Project	Group 2 - Park-It: Parking Management System
Repository cloned	https://github.com/millerdevera6-glitch/park-it-database, Branch: main
Database set up result	Success (Verified PostgreSQL 18.6 on port 5432, schema built with 5 tables, populated sample data, configured least-privilege roles).
Access-control folder link	https://github.com/Nycri/brewpos/tree/main/access-control
Access-control status	Completed
Feedback sent to owners	(05/11/2026)
PDF progress document	Activity4_Progress_Report_Group2.pdf
Last updated	2026-11-05

AI USAGE DECLARATION

Google Gemini 3.8 Flash was utilized during project evaluation to inspect database schema definitions, evaluate relational hierarchy structures, formulate role verification test scenarios, analyze database maintenance operations and query performance execution plans, verify backup creation and restoration procedures, and structure technical report documentation. All database statements, configuration checks, and verification procedures were directly executed and validated against PostgreSQL 18.6 within the local database environment.